Notes for Large Scale Drone Perception

Henrik Skov Midtiby

2026-02-16 18:58:33

Contents

1 Color segmentation 3

2 Image in 3D space 11

3 Image acquisition 12

4 Optimization 17

5 Hints 19

6 Solutions 21

Introduction

The topic of computer vision is a rather large topic, with a wealth of sources of information. Each source with a certain focus on the topic. These notes, that were written for the Large Scale Drone Perception course taught at the University of Southern Denmark, will focus on certain aspects of computer vision that we find especially relevant when analysing images from a drone.

It is not possible to cover all the topics extensively, but I have tried to provide a suitable overview of each topic and furthermore some additional resources are listed if you want to dig deeper into the topics.

Best regards,

Henrik Skov Midtiby

Recommended resources for computer vision

The following resources are highly recommended if you want to explore the topics of computer vision.

- The lecture series *First Principles of Computer Vision* presented by Shree Nayar from School of Engineering at Columbia University provides both an overview of the general topics and provides a treasure trove of details about many central algorithms
  Web: First Principles of Computer Vision
- The monographs from the course *First Principles of Computer Vision* by Shree Nayar.
  Web: First Principles of Computer Vision - Monographs
- The book *Computer vision: Algorithms and Applications* by Richard Szeliski also provides a good overview of the topic
  Web: Computer Vision: Algorithms and Applications by Richard Szeliski

Chapter 1

Color segmentation

The topic of this chapter is color segmentation, i.e. how to segment an image into different components based on color information on the pixel level. A way of representing colors are needed to do color segmentation, this is the color space, which will be discussed in section 1.1. Given a certain color representation, the next step is to make a decision rule for which colors belong to each of the classes of the segmentation. Some often used decision rules are described in section 1.2. In section 1.4 we will look at how to implement this in python using the opencv and numpy libraries.

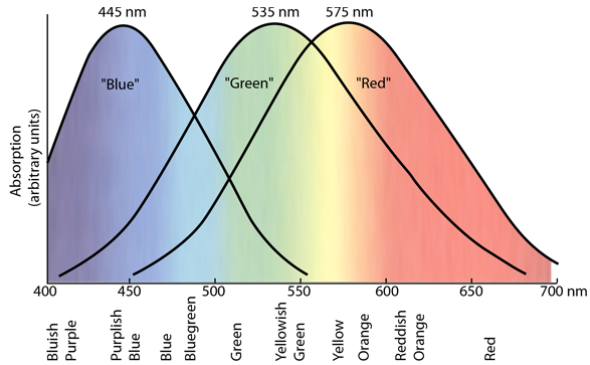

Figure 1.1: *Spectral sensitivity of the three types of cones in our eyes (Hyperphysics 2023).*

1.1 Color spaces

Digital images consist of pixels arranged in a pattern, usually a rectangular grid. Each pixel contain information about the color of that particular part of the image. How the color of a pixel is represented is denoted the *color space*. There exists many different color spaces, in this chapter the following color spaces will be discussed.

- Red, Green and Blue (RGB)
- Hue, Saturation and Value (HSV)
- CieLAB
- OK LAB

Each color space has some associated benefits and disadvantages.

1.1.1 Red, Green and blue (sRGB and linear sRGB) color space

The inspiration for the RGB color space, is how the human eyes perceive light. To sense color our eyes are equipped with three different types of *cones*, which each are sensitive to a certain range of the electromagnetic spectrum. That humans have three different types of cones, means that three color values / properties are needed to describe the colors that the human eye can perceive. The spectral sensitivity of the cones are shown in figure 1.1. By adding different amounts of red, green and blue light respectively, it is possible to generate nearly all the color that the human eye can perceive[1]. The RGB color cube is a visualization of the arrangement of colors described by the RGB color space, it is shown in figure 1.2.

In RGB, the amount of red, green and blue light that should be added to form a color is described using three numbers; often integers in the range $[0 - 255]$.

As humans are more sensitive to variations in light in dark colors, a γ (gamma) value is used to transform stored values into amounts of light using the equation

$$V_{\text{out}} = A \cdot V_{\text{in}}^{\gamma} \tag{1.1}$$

Where V_{in} is the encoded value, V_{out} is the amount of emitted light, A is a scaling factor and γ is the gamma value. The gamma value is usually around 2[2]. Two examples of a gradient between black and white is shown in figure 1.3, for the linear gradient it is very difficult to see the change in the dark colors,

1. Web: RGB color model
2. Web: Gamma correction

whereas the change is distributed more equally in the gamma corrected sGRB gradient. The nonlinear gamma correction can be problematic when doing calculations with colors[3]. In an sRGB gradient between two primary colors, the mixed colors appear to be darker than it should be. See an example in figure 1.4.

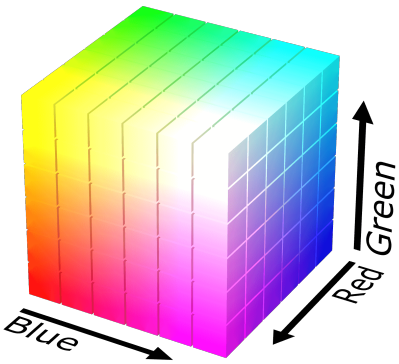

Figure 1.2: *The RGB color cube with the bright faces oriented towards the camera (Wikipedia 2023).*

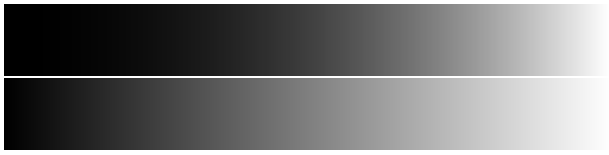

Figure 1.3: *Linear sRGB gradient (top) and gamma corrected sRGB gradient (bottom).*

Figure 1.4: *Linear sRGB gradient (top) and gamma corrected sRGB gradient (bottom).*

1.1.2 Hue, Saturation and Value / Lightning

One issue with the RGB color space is that it is very different from the way we usually describe colors, e.g.

- a dark green color
- a vibrant red color
- a pale yellow color

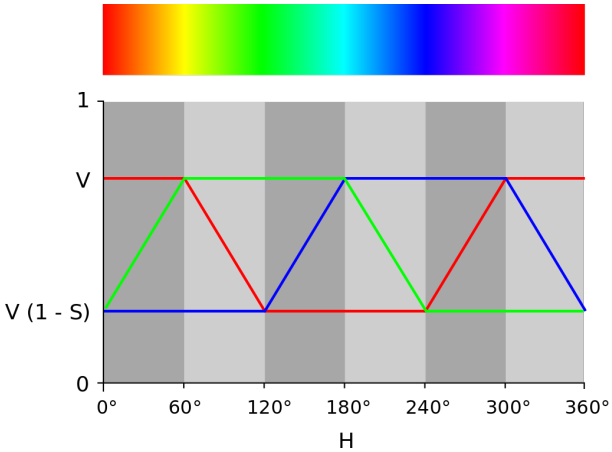

Figure 1.5: *Visual description of how to convert between HSV and RGB color representations. From wikipedia.*

In these cases a color (red, green, blue, yellow, ...) is mentioned along with a description of its brightness and saturation. The HSV color space describes colors in a very similar way. In the HSV color space a color is described in terms of the following three values[4]:

- Hue
- Saturation
- Value

Hue describe the basic color (red ~ 0, yellow ~ 60, green ~ 120, cyan ~ 180, blue ~ 240 and magenta ~ 270) as a number between 0 and 360. Hue is cyclic, which means that the hue values 1 and 359 are close to each other. Saturation is a number between 0 an 255 describing how much of the pure color specified by the hue is present in the color to describe, if the saturation is low, the color is a shade of gray and if it is high the color appear bright.

1.1.3 CIE LAB

The CIE LAB color space is an attempt at making a perceptually uniform color space, where distances in the color space matches the perceived difference between two colors as a human would interpret it. The three components of the color space are the lightness value L, green/red balance a and the blue/yellow balance b[5].

3. Video: Computer color is broken (4 min)
4. Web: HSL and HSV
5. Web: CIELAB color space

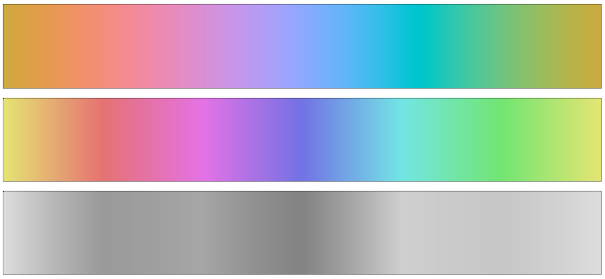

Figure 1.6: *A color gradient from OKLAB (top) compared to a similar color gradient from the HSV (middle) color space. According to the used color model, both color gradients should have constant luminosity. At the bottom is the predicted luminosity of the HSV gradient using the OK LAB color space. From (Ottosson 2023).*

1.1.4 OK LAB

OK LAB is a quite new color space, that was first described in 2020. The OK LAB color space have better numerical properties and appear more perceptually uniform than CIE LAB[6].

To explore some properties of different color spaces this interactive gradient tool is highly recommended:

- https://raphlinus.github.io/color/2021/01/18/oklab-critique.html

> For more information, see
>
> - Captain Disillusion / Color, https://www.youtube.com/watch?v=FTKPOY9MVus

1.2 Color segmentation

Given a color balue (e.g. RGB), the color based segmentation task is to determine if the color value belongs to a certain group of colors (i.e. green vegetation).

1.2.1 Independent channel thresholds

A basic approach for color based segmentation is to look at each color channel separately. E.g. if we want to see if a color is close to orange. In RGB the orange color is given by the values ($R = 230, G = 179, B = 51$). A set of requirements for a new color to be classified as orange could be the following

$$\begin{aligned} 200 < R < 255 \\ 149 < G < 209 \\ 21 < B < 81 \end{aligned}$$

To perform such a color classification the opencv function `inRange` is useful.

This approach forces the shape of the regions in the color space to accept to have a rectangular shape.

1.2.2 Euclidean distance

A different approach is to look at the the color to classify and the reference color and then calculate a distance between these. Let R_r, G_r and B_r be the reference color value (in RGB color space) and let R_s, G_s and B_s be the color sample that should be classified.

The Euclidean distance can then be calculated using the Pythagorean theorem as follows:

$$\text{distance} = \sqrt{(R_s - R_f)^2 + (G_s - G_r)^2 + (B_s - B_r)^2}$$

If the notation $C_s = [R_s, G_s, B_s]^T$ and $C_r = [R_r, G_r, B_r]^T$, the equation can be written in a more compact form

$$\text{distance} = \sqrt{(C_s - C_r)^T \cdot (C_s - C_r)}$$

The decision rule to accept a color as being close enough to a reference colors can then be written as a requirement on the maximum allowed value of the calculated distance. This decision boundary has the shape of circles in the color space.

To determine the reference color and the threshold, a number of pixels of the object to recognize can be sampled. The reference value can then be set to the mean color value of these pixel and the threshold distance can be set to the maximum distance from the mean color value to the sampled color values.

1.2.3 Mahalanobis distance

To further adapt the decision surface to a set of sampled color values, the Mahalanobis distance can be used[7].

$$\text{distance} = \sqrt{(C_s - C_r)^T \cdot S^{-1} \cdot (C_s - C_r)}$$

6. Web: A perceptual color space for image processing
7. Web: Mahalanobis distance

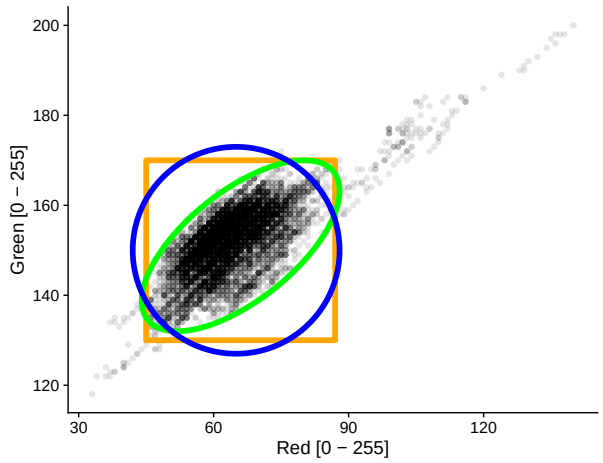

Figure 1.7: *Decision boundaries generated with the* inrange *function (orange box), the euclidean distance (blue circle) and the Mahalanobis distance (green ellipse). The plotted pixel values are sampled from an image of a patch of grass.*

where C_r is the reference color, S is a covariance matrix and C_s is the sample color.

The decision surface generated by the Mahalanobis distance is an ellipsoid in the color space.

1.2.4 Decisions boundaries

By using the described techniques for color based segmentation, a number of different regions in the color space can be delineated. These regions are visualized in figure 1.7. In the figure the red and green color values of a number of pixels are shown in a coordinate system. The boundary produced by the `inRange` function is a square depicted in orange, the boundary produced by thresholding the euclidean distance is a circle shown in blue and the boundary of thresholding the Mahalanobis distance is shown as a green ellipse.

The quality of the obtained segmentation results depend on both the chosen color space and the chosen type of decision boundary. For some tasks, tools like `inRange` works very well, and for other tasks it might be necessary to convert to a different colorspace or use a more flexible decision boundary.

1.3 Choosing a proper threshold value

For both the euclidean and the Mahalanobis distance based color segmentations, a threshold is needed to decide where to place the decision boundary between accepting the color as close enough to the reference color or not.

1.3.1 Otsu thresholding

Otsu's method is the classic approach for determining a suitable threshold to differentiate between two groups of color values. It works decently in most cases.

https://en.wikipedia.org/wiki/Otsu%27s_method

1.3.2 Generalized Histogram Thresholding

In 2020 the *Generalized Histogram Thresholding* (GHT) was introduced by Jonathan T. Barron. Generalized Histogram Thresholding performs better than Otsu's method, but uses some more involved mathematics.

The central element in the Generalized Histogram Thresholding method is the function

$$\mathrm{GHT}(\mathbf{n}, \mathbf{x}, \nu, \tau, \kappa, \omega)$$

where $\mathbf{n}$ and $\mathbf{x}$ is the histogram counts and bin centers and ν, τ, κ and ω are tuning parameters. The tuning parameter ω lets the user specify an estimated location of the threshold (as a number from zero to one) and the associated κ value is how much the provided ω value will influence the determined threshold. By varying the ν parameter from zero to infinity, the behavior of GTH changes from Minimum Error Thresholding (MET) at $\nu = 0$ to Otsu's method for $\nu \to \infty$. In between these two extremes, the GHT seems to perform better than these two methods. The following values seems to work well for many cases: $\nu = 2^5$, $\tau = 2^10$, $\kappa = 0.1$ and $\omega = 0.5$.

https://arxiv.org/abs/2007.07350

https://github.com/jonbarron/hist_thresh

1.4 Color segmentation in python

Color segmentation based on independent color channels are implemented in opencv's `inRange` func-

tion. The following example demonstrates how to use the `inRange` function

```
filename = "inputfile.jpg"
lower_limits = (130, 130, 100)
upper_limits = (255, 255, 255)
img = cv2.imread(filename)
segmented_image = cv2.inRange(img,
    lower_limits, upper_limits)
```

To extract a sample of pixels from an image, it is effective to annotate a copy of the image with a unique color (e.g. pure red (255, 0, 0)) in an image editing program like Gimp. Then the location of the annotated pixels can be determined with `inRange` and an image mask can be generated. Given the mask, the function `meanStdDev` can be used to calculate the mean and standard deviation of the color values in the original image.

```
file = "image.jpg"
file_annot = "image_annotated.jpg"
img = cv2.imread(file)
img_annot = cv2.imread(file_annot)
lower_limit = (0, 0, 245)
upper_limit = (10, 10, 256)
mask = cv2.inRange(img_annot,
    lower_limit, upper_limit)
mean, std = cv2.meanStdDev(
    img, mask = mask)
```

As there is no builtin function for calculating the covariance matrix, we need to extract the pixel values into a list (here named pixels) and then select the observations with the obtained mask. The `reshape` function is used to change the image dimensions to an array with one row per pixel and three columns with the associated color values. The average pixel value and the covariance matrix can then be found as follows using the `np.cov` and `np.average` functions:

```
pixels = np.reshape(img, (-1, 3))
mask_pixels = np.reshape(mask, (-1))
annot_pix_values = pixels[mask_pixels == 255, ]
avg = np.average(annot_pix_values, axis=0)
cov = np.cov(annot_pix_values.transpose())
```

Often it can be beneficial to perform further analysis of the pixel values (ie. to visualize them). To save the pixel values to a file, this code can be used:

```
np.savetxt("annotated_pixel_values.csv",
        annot_pix_values,
        delimiter=",",
        fmt="%d")
```

To visualize the distribution of the extracted color values, the python package `matplotlib` can be used as follows

```
import matplotlib.pyplot as plt
fig, ax1 = plt.subplots()
ax1.plot(pixels[:, 1], pixels[:, 2], '.')
ax1.set_title('Color values of a patch of grass')
plt.xlabel("Green [0 - 255]")
plt.ylabel("Red [0 - 255]")
fig.tight_layout()
plt.savefig("color_distribution.pdf", dpi=150)
```

To calculate the squared Euclidean distance to a reference color, the following code can be used:

```
shape = pixels.shape
avg_value = np.repeat([avg],
    shape[0], axis=0)
diff = pixels - avg_value
dotproduct = diff * diff
euc_dist = np.sum(dotproduct, axis=1)
euc_dist_image = np.reshape(euc_dist,
        (img.shape[0], img.shape[1]))
```

The code is based on vectorized operations of the pixel arrays. This is much faster (one or two orders of magnitude) than doing the calculation pixel by pixel[8]. To enable the vectorized calculations, the image data needs to be reshaped to a $(n \times 3)$ matrix to do the distance calculation efficient and then back to the original image dimensions.

Similarly the squared Mahalanobis distance can be computed with the code:

```
inv_cov = np.linalg.inv(cov)
moddotproduct = diff * (diff @ inv_cov)
mahalanobis_dist = np.sum(moddotproduct,
    axis=1)
mahalanobis_distance_image = np.reshape(
    mahalanobis_dist,
      (img.shape[0],
       img.shape[1]))
```

In the shown code, two different kinds of matrix multiplications are used. `*` is used to do point wise multiplication of two matrices (that have the same shape) and `@` is used to perform regular matrix multiplication.

1.5 Reference to python and OpenCV

It is highly recommended that you create a virtual python environment that you can work within when working on exercises in this course.

8. This is case in Python, if you use C / C++ this is not an issue.

- Web: Managing Python Projects With uv: An All-in-One Solution

Some references on how to use OpenCV in python:

- Web: Getting started with images
- Web: Basic Drawing
- Web: Operations with images
- Web: Changing Colorspaces
- Web: Basic Thresholding Operations

1.6 Getting started exercises

To get access to the support files for these exercises, do the following

- Download the zip folder with exercises from Its Learning and unzip it in a suitable location
- Open a command line and enter the directory containing the unpacked exercises
- Run the code for the first exercise as follows

```
cd 01_getting_started/solutions
uv run python ex01_draw_on_empty_image.py
```

You should now have all the required dependencies installed in a virtual environment. To run python programs from within the virtual environment use the following code

```
uv run python name_of_python_script.py
```

Each group of exercises are placed in a directory. The exercises described in this section are in the `01_getting_started` directory.

Exercise 1.6.1

Put the following content into a file named *draw_on_empty_image.py*

```
import numpy as np
import cv2
img = np.zeros((100, 200, 3), np.uint8)
cv2.line(img, (20, 30), (40, 120),
         (0, 0, 255), 3)
cv2.imwrite("test.png", img)
```

Run the following command on the command line

```
uv run python draw_on_empty_image.py
```

If everything worked, there should now be an image named "test.png" in the directory containing a black canvas with a thick red line drawn on top.

Exercise 1.6.2 hint

Load an image into python / opencv, draw something on it and save it again.

Exercise 1.6.3 hint

Load an image and save the three color channels (RGB) in separate files.

Exercise 1.6.4 hint

Load an image and save the upper half of the image in an file.

Exercise 1.6.5 hint

Load an image, convert it to HSV and save the three color channels.

Exercise 1.6.6 hint

Load an image. Locate the brightest pixel in that image and draw a circle around that pixel.

Exercise 1.6.7 hint

Load the image `flower.jpg`[9] and generate a pixel mask (a black and white image) of where the image contains yellow pixels. Use color information to determine the location of the pixel mask. Save the pixel mask to a file. Experiment with using different color spaces.

Exercise 1.6.8 hint

Load the image from exercise 1.6.7. Plot the amount of green in each pixel in a single row of the image. Use matplotlib to visualise the data. Repeat this for the two other color channels.

Exercise 1.6.9 hint hint

Load an image. Use matplotlib to visualise a histogram of the pixel intensities in the image.

Exercise 1.6.10

Calculate the average RGB pixel values of the flower petals in the image `flower.jpg`. You can use the red regions in `flower-petals-annotated.jpg` as a mask.

Exercise 1.6.11

Calculate the euclidean distance in the RGB color space from the reference color $(178, 180, 187)$ to all pixels in the image from exercise 1.6.7.

9. The image is from wikimedia: https://commons.wikimedia.org/wiki/File:Daubeny%27s_water_lily_at_BBG_(50824).jpg

Exercise 1.6.12

Plot the histogram of the euclidean distance image from exercise 1.6.11.

Exercise 1.6.13

Use the Otsu method for determining the threshold to segment the distance image in exercise 1.6.11.

Exercise 1.6.14

Use the Generalized Histogram Thresholding method for determining the threshold to segment the distance image in exercise 1.6.11.

1.7 Counting brightly colored objects

These exercises are located in the directory `02_counting_bright_objects` in the git repository with exercises. These exercises will be based on some sample images of colored plastic balls on a grass field. To download the images, run the command `make download_images` in the directory.

You should complete the three exercises with at least one under exposed image, one well exposed and one over exposed image.

To get access to the images, do the following

- Enter the subdirectory `02_counting_bright_objects/input`
- Run the command `make download_images`

The code snippet shown in figure 1.8 might be handy for debugging the segmentation exercises.

Exercise 1.7.1 hint hint

For each image locate pixels that is a) saturated in all color channels (R = G = B = 255) and b) saturated in at least one color channel (max(R, G, B) = 255). In addition count the number of saturated pixels according to the two measures.

Exercise 1.7.2 hint

We want to count the number of colored balls in the images. As a first step towards that goal, segment the images in the RGB color space. You are only allowed to look at the color channels individually (eg. $R \geq 55$) or look at linear combinations of the color channels ($G - R \geq -10$). Which of the three images are easiest to segment?

Exercise 1.7.3 hint

Segment the images in the HSV color space. You are only allowed to look at the color channels individually (eg. $H \geq 33$) or look at linear combinations of the color channels ($V - S \geq 0.2$). Which of the three images are easiest to segment?

Exercise 1.7.4 hint

Segment the images in the CieLAB color space. You are only allowed to look at the color channels individually (eg. $L \geq 33$) or look at linear combinations of the color channels ($a - b \geq 0.2$). Which of the three images are easiest to segment?

Exercise 1.7.5 hint

Choose one of the segmented images, filter it with a median filter of an appropriate size and count the number of balls in the image.

Exercise 1.7.6 hint hint

Use the python package `exifread` to extract information about the gimbal pose (yaw, pitch and roll) from one of the images.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def compare_original_and_segmented_image(original, segmented, title):
    plt.figure(figsize=(9, 3))
    ax1 = plt.subplot(1, 2, 1)
    plt.title(title)
    ax1.imshow(original)
    ax2 = plt.subplot(1, 2, 2, sharex=ax1, sharey=ax1)
    ax2.imshow(segmented)

img = cv2.imread("input/under_exposed_DJI_0213.JPG")
segmented_image = cv2.inRange(img, (60, 60, 60), (255, 255, 255))
compare_original_and_segmented_image(img, segmented_image, "test")
plt.show()
```

Figure 1.8: *Codesnippet for comparing images next to each other in python, eg. an input image and a segmented image.*

Chapter 2

Image in 3D space

2.1 Exercises

Exercise 2.1.1

The folder `2026_LSDP_Camera_in_3d_space.zip` contains:

- `IMG` folder - 39 undistorted (distortion removed and principal point moved to the middle of an image) images
- `camera.xml` - calibration file of a camera, including focal length, principal point and distortion parameters
- `EP.txt` - Extrinsic parameters of the images (XYZ and rotation matrix).

Choose two images with overlap, find corresponding point on them and perform triangulation together with quality assessment in form of reprojection error.

Note, in OpenCV we can use function

```
cv.triangulatePoints(
    P_1,
    P_2,
    points_1,
    points_2)
```

- Matrix P is calculated as $K[R|t]$ where
  - K - camera matrix, in case of undistorted images and just one focal length it will be $[[f, 0, w/2], [0, f, h/2], [0, 0, 1]]$
  - R - rotation matrix unchanged,
  - t - translation vector from camera to world so, $t = -R[[X], [Y], [Z]]$
- As a result you will get homogenous coordinates, so you need to normalize to get correct position (divide all by last element of the vector).

To reproject we use equation:

$$point = P[[X], [Y], [Z], [1]]$$

again, as a result we get homogenous points.

To get reprojection error calculate the distance between your observation (pixel coordinates) and reprojected position (after normalization like above).

Exercise 2.1.2

Again, choose two images. For one select one (or multiple points). Using provided data calculate fundamental matrix F and then calculate and draw epipolar lines for selected points on the second image.

Note, Fundamental matrix is calculated for a specific direction. We use equation:

$$F_{1-2} = \left(K^{-1}\right)^T \; [t_\times] \; R \, K^{-1}$$

where:

- R - rotation from 1 to 2, so $R = R_2 \, R_1^T$
- t - translation vector from 1 to 2, so $t = R_2([[X_1 - X_2], [Y_1 - Y_2], [Z_1 - Z_2]])$

As a result you will get lines as a, b, c from $ax + by + c = 0$.

Exercise 2.1.3

Take two images of the same scene from different perspective. Locate 15 corresponding point pairs and save their coordinates to a file.

Exercise 2.1.4

Calculate the fundamental matrix using the function

```
cv.findFundamentalMat
```

then draw the points (from first image) and the corresponding epipolar lines on the second image.

Repeat the exercise for the opposite direction.

Chapter 3

Image acquisition

When capturing images there is a set of settings that needs to be chosen. These settings determines the quality of the acquired images and is therefore important to consider.

The image processing pipeline consists of multiple steps, as visualized in figure 3.1. Most courses in machine vision and image analysis will only look at the last step of the pipeline. If you are able to adjust the image acquisition it can make the following image processing much easier.

This chapter will look at the typical camera settings that can be adjusted before or during image acquisition. Some often seen image artefacts will also be discussed.

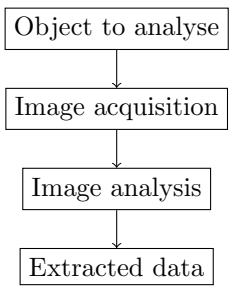

Figure 3.1: *The image processing pipeline. Most courses on machine vision focus on the last two steps, whereas the LSDP course also considers the image acquisition step.*

3.1 Image formation

Lets first look at how images are acquired in a digital camera. An array of light sensitive circuits are used to convert light into electric signals, more about that in section 3.2.

To get a good image, light from the object we want to photograph should reach the right part of the sensor. If the imaging sensor is directed towards object, light from one point of on the object will reach multiple locations on the sensor. This means that all pixels

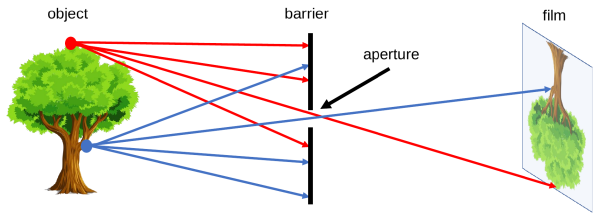

Figure 3.2: *Image formation using the pinhole camera model. From (Hata and Savarese 2021).*

on the sensor will receive approximately the same amount of light and thus an extremely blurry image will be acquired.

For improving the image quality, the next step is to hide the sensor in a box with a tiny opening (a pinhole) in front of the photosensitive side of the sensor. This way light from the one point in the scene will only be able to reach a single pixel on the sensor. There is however one big issue with this approach, the amount of light that reaches the sensor is very low, as it has to pass through the pinhole. The amount of light that is let through can be increased by enlarging the hole, but that would blur the formed image.

Somehow we need to focus light from the scene onto the sensor. By placing a converging lens in front of the opening in the box, light from a certain distance can be focused on the imaging sensor. In some cameras there is an adjustable aperture mounted next to the lens, that allows the user to adjust how much light that reaches the sensor.

> This note provides an overview of the pinhole camera model.
> Web: CS231A Course Notes 1: Camera Models

3.2 A simple image sensor

A digital camera is built around a device that is able to measure properties of the received light. This device is the imaging sensor. Today the Complementary Metal Oxide Sensor (CMOS) is the most commonly used[1]. Earlier the Charge Coupled Device (CCD) was a suitable alternative to CMOS sensors. Before 2020 CCD's provided better signal to noise levels than CMOS sensors, and were preferred in astronomical telescopes.

The imaging sensor is able to count the number of photons that each pixel in the sensor receive. The pixels are arranged in a rectangular pattern. In combination with the lens, this enables the sensor to measure the number of photons coming from different directions[2].

The properties that can be adjusted in a camera system are the following[3] [4]:

- the aperture, which limits the amount of light that passes through the lens
- the shutter time, in which the sensor registers new photons
- the ISO sensor sensitivity, which determines the gain of the sensor, how many photons can be received before the sensor is saturated?

It depends on the situation, but usually the settings are specified in the following order

1. shutter time, high values are preferred, but motion blur should usually be avoided
2. aperture, high values are preferred, if it is difficult to get the object in focus the aperture can be lowered
3. ISO, choose such that the exposure of the image is as desired. This can be checked through a histogram.

Personally I prefer to have images slightly underexposed.

3.2.1 Rolling and global shutter

There are also two different ways of reading out data from image sensors, rolling shutter and global shutter. In an imaging sensor with global shutter, the entire image is exposed simultaneously. After exposure all pixel values are read out. This makes it easier to interpret the images and they do not suffer from distortions based on motion of either the camera or the object in the scene. The main disadvantage of global shutter is that the frame rate is reduced compared to a rolling shutter sensor.

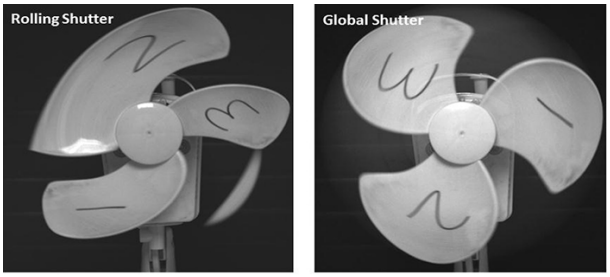

Figure 3.3: *Illustration of artefacts from using a rolling shutter camera. From (Oxford Instruments 2023).*

In a rolling shutter sensor, one row of pixels is read out at a time. This means that the acquired image contains elements exposed at different times; this causes deformation of objects in motion[5].

> Additional resources on rolling and global shutter
>
> - Web: Global & rolling shutter

3.2.2 Camera settings when capturing video

Special consideration should be taken when capturing video. It can be desirable to have some amount of motion blur in the recorded video, which ideally should match the used frame rate. It is recommended to use shutter speed of 50% of the time between adjacent frames when capturing video. Eg. when capturing video with 30 fps, the shutter speed should be around 1 / 60. To achieve a suitable exposure, it is sometimes needed to mount a *neutral density* filter in front of the camera, to reduce the amount of light that enters the camera. If much lower, the motion blur does not match what we will expect and the frames would look choppy[6].

1. Web: Active pixel sensor
2. Web: What camera measure
3. Web: A Beginner's Guide to Aperture, Shutter Speed, and ISO
4. Video: Aperture, Shutter Speed, ISO, & Light Explained ... (14 min)
5. Video: Why Do Cameras Do This? (Rolling Shutter Explained) (7 min)
6. Web: Frame rate vs. shutter speed, setting the record straight

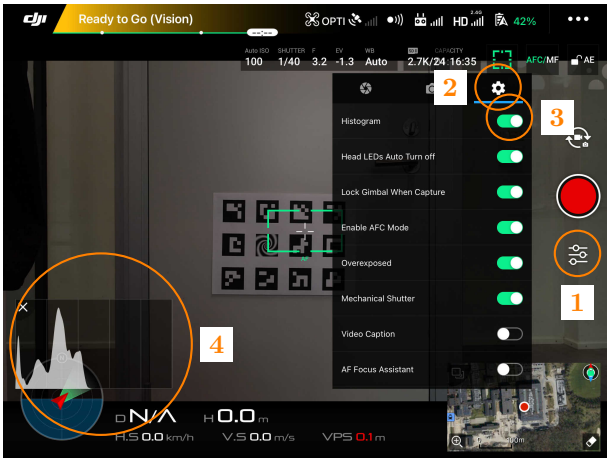

Figure 3.4: *How to enable the histogram in DJI Go 4. Open camera settings (1), gear (2), enable histogram (3) and the histogram is now visible (4).*

3.2.3 Adjusting camera settings on a DJI drone

When capturing images with a DJI drone, it is possible to adjust the camera settings while the drone is in the air. To get an idea about the exposure with the current camera settings, it is helpful to enable the histogram in DJI 4 Go (see figure 3.4.

It is also possible to control the camera settings (see figure 3.5). The camera modes that are available are *Auto*, *Aperture priority*, *Shutter priority* and *Manual*. Usually you can get decent results by setting the camera on auto and then adjust the exposure. Most of my flights are conducted with the exposure set to EV $= -1.3$.

- Video: Drone Mapping: Best Shutter Speed Settings for Perfect Results

3.3 Optical filters

Linear polarization can be used to remove reflections from vertical (e.g. a window) or horizontal surfaces (e.g. a sea surface) [7].

A band pass filter can be used to only allow a certain part of the electromagnetic spectrum to reach the sensor. High pass and low pass filters also exist.

3.4 Shape of the aperture

The shape of the aperture determines how bright object that are not in focus appear in the image [8].

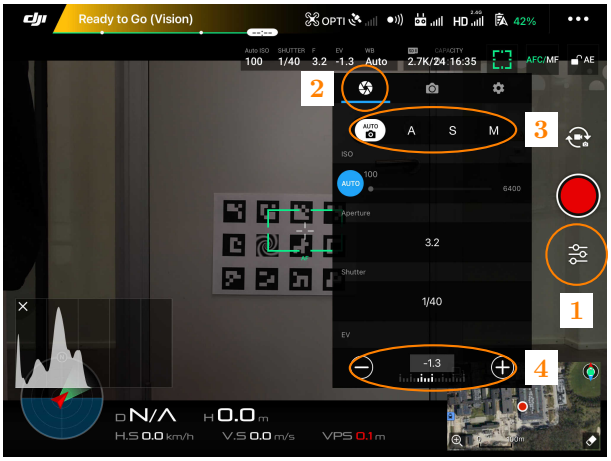

Figure 3.5: *Finding the camera settings in DJI Go 4. Settings (1), camera (2), camera mode (3), exposure (4).*

3.5 Pinhole camera model

The pinhole camera model is used for mapping 3D world coordinates to 2D image coordinates as follows:

$$s \cdot \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \cdot \left[R \,\middle|\, \mathbf{t} \right] \cdot \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} \tag{3.1}$$

The pinhole model maps the 3D world coordinate (given in homogeneous coordinates) $(X, Y, Z, 1)^T$ to 2D image coordinates (also in homogeneous coordinates). To compute the mapping, the location and orientation of the camera is used. This is specified as a position vector $\mathbf{t}$ and rotation matrix R of the optical centre of the camera. Some parameters related to the optical system is encoded K matrix. The included parameters cover the focal length in the x and y-directions (f_x and f_y) and the location of the axis of the optical system (c_x and c_y) as well as the parameter describing the skew γ between the x and y axes of the camera sensor. On modern cameras the skew parameter can usually be set to zero.

$$K = \begin{pmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$$

- Web: Geometry of image formation

7. Web: Polarizing filter (photography)
8. Video: A filter with holes in one line

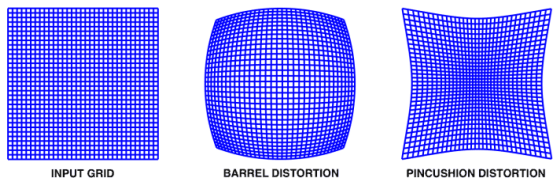

Figure 3.6: *Examples of radial lens distortions. From (Sadekar 2023).*

3.6 Lens distortion

The pinhole model introduced in 3.5 makes the assumption that all light that reaches the camera sensor passes through a single point – the pin hole. However this is usually replaced with an optical system, consisting of one or more lenses. Such an optical system will introduce distortions to the formed image. These distortions can be divided into two groups

1. radial distortions
2. tangential distortions

The radial distortions are introduced by the optical system, and can be modelled with the equations:

$$r = \sqrt{(x - x_c)^2 + (y - y_c)^2} \tag{3.2}$$

$$x_{\text{distorted}} = x \cdot (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \tag{3.3}$$

$$y_{\text{distorted}} = y \cdot (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \tag{3.4}$$

where r is the distance to the optical axis of the system. The tangential distortions can similarly be modelled with the equations.

$$x_{\text{distorted}} = x + [2p_1 xy + p_2(r^2 + 2x^2)] \tag{3.5}$$

$$y_{\text{distorted}} = y + [p_1(r^2 + 2y^2) + 2p_2 xy] \tag{3.6}$$

In most cases the three parameters (k_1, k_2 and k_3) modelling the radial distortions is enough to describe the distortions. In a few cases it could make sense to use additional parameters if the distortion is hard to model; an example of this are fish eye lenses.

- Web: Camera calibration With OpenCV
- Web: Understanding lens distortion
- Web: Camera Calibration and 3D Reconstruction – Detailed description

3.7 Camera calibration

Camera calibration is the process of estimating both the parameters in the pinhole camera model and the parameters in the lens distortion model.

- Web: Camera calibration using OpenCV
- Web: Camera calibration with large chessboards

3.8 Projecting observations on to the ground plane

If we look at objects at a known altitude (ie. the sea surface with $Z = 0$) compared to the camera and also know the camera position and orientation, it is possible to calculate the 3D world coordinates of the point we are looking at. So given the image coordinates u and v, the task is to estimate the X and Y parts of the real world coordinates, provided this equation.

$$s \cdot \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \cdot [R \,|\, \mathbf{t}] \cdot \begin{pmatrix} X \\ Y \\ 0 \\ 1 \end{pmatrix} \tag{3.7}$$

3.9 Collection of data

There is a lot of things to consider prior to going out on a mission to acquire data with a drone. To ensure that the mission can be completed successfully, checklists can be used to ensure that you take care of things in the right order. When operating UAVs checklists are used to ensure the safety and success of the mission that is to be conducted.

During prototype tests with a B17 bomber in 1935, the pilots failed to configure the plane properly, which caused the plane to crash. To avoid similar problems in the future the company behind the plane, Boing, introduced preflight checklists to ensure that the plane was configured correctly prior to takeoff [9].

If you want to read more about checklists and their adoption in aviation this source could be of interest Web: Not flying by the book: Slow adoption of checklists and procedures in WW2 aviation

Exercise 3.9.1

Discuss what actions you would take before taking off with a multirotor UAV. Then make a pre flight checklist for a multirotor UAV.

9. Web: Checklist born – B-17 Bomber pre-flight checklist

Exercise 3.9.2

Discuss what actions you would take after landing with a multirotor UAV. Then make a post flight checklist for a multirotor UAV.

Exercise 3.9.3

Collect images with different exposures

- Under exposed images
- Well exposed images
- Over exposed images

Collect images with different orientations of the camera

- Change yaw or pitch

Exercise 3.9.4

Take a look at some of the acquired images (eg. `DJI_0001.JPG` and `DJI_0177.JPG`). Extract information from the image about the position of the UAV during image capture and the orientation of the camera. Try to verify the information by locating known objects in the images.

Exercise 3.9.5 hint

A set of images with severe motion blur have been acquired and you have to redo the flight. Which camera settings would you modify for the flight?

Exercise 3.9.6

Take the image DJI_0177.JPG and find information about the camera orientation in the exif data that is embedded in the image. Modify the camera projection script (provided as matlab code) to match the used camera orientation.

Exercise 3.9.7 hint

Project the image from exercise 3.9.6 down on the ground plane.

Exercise 3.9.8

Take multiple images and project them down onto the same ground plane. The images should be taken with the UAV in the same position but with differences in the gimbal orientation.

Loose ends

- Web: Quick D: Static helicopter blades
- Web: Understanding lenses, aperture, focal length and fisheye
- Web: CD / Shutter Speed

Chapter 4

Optimization

In this chapter we will look at how so solve a certain type of optimization problems, namely *non-linear least squares* problems. The problem consists of finding the value of the parameter vector $\vec{\beta}$, that minimizes the expression.

$$S\left(\vec{\beta}\right) = \sum_{i=1}^{m} \left[y_i - f\left(x_i, \vec{\beta}\right)\right]^2$$

where $(y_i, \vec{x}_i)$ are observations of dependent and independent variables of a non-linear model $y = f(\vec{x})$.

The ability to solve this type of problems will allow us to solve the Perspective-n-Point (PnP) problem, in which the location and orientation of a camera should be estimated from a number of features in an image, the features 3D world coordinates and the internal camera parameters.

To solve the non-linear least squares optimization problem, we will look at the following methods:

- gradient descent
- Gauss Newton algorithm
- Levenberg Marquardt algorithm

Additional resources

- Levenberg–Marquardt algorithm - Wikipedia
- The Levenberg-Marquardt Algorithm - Ananth Ranganathan 2004
- Numerical Recipes - 15.5 Nonlinear Models
- The Levenberg-Marquardt algorithm for nonlinear least squares curve-fitting problems - Gavin 2026
- Pose estimation - OpenCV
- Perspective-n-Point (PnP) pose computation - OpenCV

4.1 Gradient descent univariate functions

Exercise 4.1.1 solution

Plot the function $f(x) = \sin(x)$ over the interval $[-10, 10]$ using matplotlib.

Exercise 4.1.2 hint

Plot the function $f(x) = \sin(x)$ over the interval $[-10, 10]$ and plot the function $g(x) = x - 0.5 \cdot f'(x)$ in the same coordinate system.

How does the function $g(x)$ act close to local minima of $f(x)$?

Exercise 4.1.3 hint

Repeat exercise 4.1.2 but avoid differentiating the function by hand.

Exercise 4.1.4

Experiment with a few different functions for $f(x)$.

Exercise 4.1.5 hint hint

Lets now change the definition of the function g so it becomes

$$g(x) = x - k \cdot f'(x)$$

Experiment with the value k, how flat can you make the regions of $g(x)$ around the local minima?

4.2 Newtons method for univariate functions

Exercise 4.2.1 hint solution

Plot the function $f(x) = \sin(x)$ and plot the function

$$g(x) = x - \frac{f'(x)}{f''(x)}$$

in the same coordinate system.

How does the function $g(x)$ act close to local minima of $f(x)$?

Exercise 4.2.2 solution

What happens with $g(x)$ around local maxima of $f(x)$?

Exercise 4.2.3

Add a damping factor to $g(x)$, so the function becomes

$$g(x) = x - \frac{f'(x)}{f''(x) + k}$$

If $k = 0.9$ how does that affect the function $g(x)$?

Exercise 4.2.4

Experiment with different values of k and see how that affects the plot. Please also try to change the function $f(x)$, to see if your earlier observations still holds with other functions.

4.3 Gradient descent with multivariate functions

Exercise 4.3.1 hint

Plot the Rosenbrock banana function with parameters $a = 1$ and $b = 100$. The function is given by the equation

$$f(x) = (a - x)^2 + (y - x^2)^2$$

Exercise 4.3.2 hint

For a given point (x_k, y_k) on the Rosenbrock function, approximate the gradient of f a numerical approach.

Normalise the gradient and move a distance k against the gradient.

4.4 Pose estimation from planar marker

Exercise 4.4.1

Given a calibrated camera and the location of four known points in an image, estimate the camera pose by minimizing the reprojection error.

Use the following internal camera matrix

$$\begin{pmatrix} 1622.5 & 0. & 1007.9 \\ 0. & 1622.7 & 748.4 \\ 0. & 0. & 1. \end{pmatrix}$$

Use the image named `aruco_marker_scaled.jpg` for this task. It can be found in the `exercises.zip` file on Its Learning. The side length of the square is 513 mm.

Chapter 5

Hints

First hint to 1.6.2 Back to exercise 1.6.2

Use the methods `cv2.imread`, `cv2.imwrite` and `cv2.circle`.

First hint to 1.6.3 Back to exercise 1.6.3

Look at numpy indexing.

First hint to 1.6.4 Back to exercise 1.6.4

Look at numpy indexing.

First hint to 1.6.5 Back to exercise 1.6.5

Look at `cv2.cvtColor`

First hint to 1.6.6 Back to exercise 1.6.6

Look at `cv2.minMaxLoc`

First hint to 1.6.7 Back to exercise 1.6.7

Look at `cv2.inRange`.

First hint to 1.6.8 Back to exercise 1.6.8

An example of how to plot values with matplotlib is given here.

```
import matplotlib.pyplot as plt
plt.plot([8, 3, 6, 2])
filename = "outputfile.png"
plt.savefig(filename)
```

First hint to 1.6.9 Back to exercise 1.6.9

Look at the `plt.hist` from matplotlib.

First hint to 1.7.1 Back to exercise 1.7.1

The `cv2.inRange` function can be used for a).

First hint to 1.7.2 Back to exercise 1.7.2

Open the images in gimp and inspect the color values of the balls with the chosen color.

First hint to 1.7.3 Back to exercise 1.7.3

Open the images in gimp and inspect the color values of the balls with the chosen color.

First hint to 1.7.4 Back to exercise 1.7.4

You will need a tool to convert from RGB to CIELAB values.

First hint to 1.7.5 Back to exercise 1.7.5

199 balls was taken to the field. Do not expect to see them all in an image.

First hint to 1.7.6 Back to exercise 1.7.6

The function `exifread.process_file` is a good place to start.

First hint to 3.9.5 Back to exercise 3.9.5

Exposure time should be decreased while ISO or aperture increased to compensate for the reduction in light on the sensor.

First hint to 3.9.7 Back to exercise 3.9.7

OpenCV hint: Look at the functions: getPerspectiveTransform and warpPerspective.

First hint to 4.1.2 Back to exercise 4.1.2

The derivative of $f(x)$ can be calculated by hand.

First hint to 4.1.3 Back to exercise 4.1.3

The method `np.diff` can be helpful.

First hint to 4.1.5 Back to exercise 4.1.5

For $f(x) = \sin(x)$, the value $k = 1$ makes the function $g(x)$ flat around the local minima.

First hint to 4.2.1 Back to exercise 4.2.1

Both the first and second derivative of $f(x)$ can be calculated by hand.

First hint to 4.3.1 Back to exercise 4.3.1

```
import matplotlib.pyplot as plt
import numpy as np

def rosenbrock(x, y):
    a = 1
```

```
    b = 100
    return (a - x) ** 2 + b * (y - x**2) ** 2

delta = 0.01
x = np.arange(-4.0, 4.001, delta)
y = np.arange(-3.0, 3.001, delta)
X, Y = np.meshgrid(x, y)
Z = rosenbrock(X, Y)
levels = 0.1 * 2 ** np.arange(0, 18, 1)
plt.contour(X, Y, Z, levels)
plt.show()
```

First hint to 4.3.2 Back to exercise 4.3.2

Recall the definition of the derivative

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

Second hint to 1.6.9 Back to exercise 1.6.9

The `np.reshape` function can also be handy.

Second hint to 1.7.1 Back to exercise 1.7.1

You can use `cv2.inRange` to find all pixels that are not saturated by using suitable limits. Invert the generated mask to find the partially saturated pixels.

Second hint to 1.7.6 Back to exercise 1.7.6

The `parseString` from `from xml.dom.minidom` is also handy.

Second hint to 4.1.5 Back to exercise 4.1.5

For $f(x) = \exp(x) + \exp(-x)$, a k value around 50 makes the function $g(x)$ flat around the local minima.

Chapter 6

Solutions

Solution to 4.1.1 Back to exercise 4.1.1

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(-10.0, 10.0, 0.01)
y = np.sin(x)
plt.plot(x, y)
plt.show()
```

Solution to 4.2.1 Back to exercise 4.2.1

The function $g(x)$ appears flat close to the local minima of $f(x)$.

Solution to 4.2.2 Back to exercise 4.2.2

It is also flat.

Bibliography

Aanæs, Henrik (2015). *Lecture Notes on Computer Vision*.

Bishop, Christopher M (2007). *Pattern Recognition and Machine Learning (Information Science and Statistics)*. 1st ed. Springer. ISBN: 0387310738. URL: http://www.amazon.com/Pattern-Recognition-Learning-Information-Statistics/dp/0387310738?SubscriptionId=13CT5CVB80YFWJEPWS02&tag=ws&linkCode=xm2&camp=2025&creative=165953&creativeASIN=0387310738.

Chien, Hsiang-Jen et al. (Nov. 2016). "When to use what feature? SIFT, SURF, ORB, or A-KAZE features for monocular visual odometry". In: *2016 International Conference on Image and Vision Computing New Zealand (IVCNZ)*. IEEE, pp. 1–6. ISBN: 978-1-5090-2748-4. DOI: 10.1109/IVCNZ.2016.7804434.

Csurka, Gabriella et al. (2004). "Visual Categorization with Bags of Keypoints". In: *In Workshop on Statistical Learning in Computer Vision, ECCV*, pp. 1–22. ISBN: 9780335226375. arXiv: arXiv:1210.1833v2. URL: http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.72.604.

Garrido-Jurado, S. et al. (June 2014). "Automatic generation and detection of highly reliable fiducial markers under occlusion". In: *Pattern Recognition* 47.6, pp. 2280–2292. ISSN: 0031-3203. DOI: 10.1016/j.patcog.2014.01.005.

Hartley, Richard (2004). *Multiple view geometry in computer vision*. Cambridge, UK New York: Cambridge University Press. ISBN: 978-0521540513.

Hata, Kenji and Silvio Savarese (2021). *CS231A Course Notes 1: Camera Models*. URL: https://web.stanford.edu/class/cs231a/course_notes/01-camera-models.pdf.

Hyperphysics (2023). *The Color-Sensitive Cones*. URL: http://hyperphysics.phy-astr.gsu.edu/hbase/vision/colcon.html (visited on 01/25/2023).

KaewTraKulPong, P. and R. Bowden (2002). "An Improved Adaptive Background Mixture Model for Real-time Tracking with Shadow Detection". In: *Video-Based Surveillance Systems*. Boston, MA: Springer US, pp. 135–144. DOI: 10.1007/978-1-4615-0913-4_11.

Karami, Ebrahim, Siva Prasad, and Mohamed Shehata (Oct. 2017). "Image Matching Using SIFT, SURF, BRIEF and ORB: Performance Comparison for Distorted Images". In: *CoRR* abs/1710.0. arXiv: 1710.02726. URL: http://arxiv.org/abs/1710.02726.

Lowe, D.G. (1999). "Object recognition from local scale-invariant features". In: *Proceedings of the Seventh IEEE International Conference on Computer Vision*. IEEE, 1150–1157 vol.2. ISBN: 0-7695-0164-8. DOI: 10.1109/ICCV.1999.790410.

Lowe, David G. (Nov. 2004). "Distinctive Image Features from Scale-Invariant Keypoints". In: *International Journal of Computer Vision* 60.2, pp. 91–110. ISSN: 0920-5691. DOI: 10.1023/B:VISI.0000029664.99615.94.

Ottosson, Björn (2023). *A perceptual color space for image processing*. URL: https://bottosson.github.io/posts/oklab/ (visited on 01/25/2023).

Oxford Instruments (2023). *Rolling Shutter vs Global Shutter sCMOS Camera Mode*. URL: https://andor.oxinst.com/learning/view/article/rolling-and-global-shutter (visited on 03/07/2023).

Romero-Ramirez, Francisco J., Rafael Muñoz-Salinas, and Rafael Medina-Carnicer (2018). "Speeded up detection of squared fiducial markers". In: *Image and Vision Computing* 76, pp. 38–47. ISSN: 02628856. DOI: 10.1016/j.imavis.2018.05.004.

Sadekar, Kaustubh (2023). *Understanding Lens Distortion*. URL: https://learnopencv.com/understanding-lens-distortion/ (visited on 03/07/2023).

Sagitov, Artur et al. (May 2017). "Comparing fiducial marker systems in the presence of occlusion". In: *2017 International Conference on Mechanical, System and Control Engineering (ICMSC)*. IEEE, pp. 377–382. ISBN: 978-1-5090-6530-1. DOI: 10.1109/ICMSC.2017.7959505.

Scaramuzza, Davide and Friedrich Fraundorfer (Dec. 2011). "Visual Odometry: Part I: The First 30 Years and Fundamentals". In: *IEEE Robotics & Automation Magazine* 18.4, pp. 80–92. ISSN: 1070-9932. DOI: 10.1109/MRA.2011.943233.

Sivic and Zisserman (2003). "Video Google: a text retrieval approach to object matching in videos". In: *Proceedings Ninth IEEE International Conference on Computer Vision*. Vol. 2. Iccv. IEEE, 1470–1477 vol.2. ISBN: 0-7695-1950-4. DOI: 10.1109/ICCV.2003.1238663.

Wikipedia (2023). *RGB color solid cube.png*. URL: https://commons.wikimedia.org/wiki/File:RGB_color_solid_cube.png (visited on 01/25/2023).

Xiang Zhang, S. Fronz, and N. Navab (2002). "Visual marker detection and decoding in AR systems: a comparative study". In: *Proceedings. International Symposium on Mixed and Augmented Reality*. IEEE Comput. Soc, pp. 97–106. ISBN: 0-7695-1781-1. DOI: 10.1109/ISMAR.2002.1115078.